Università degli Studi di Palermo

dipartimento di ingegneria unipa

Introduzione a scikit-learn

INTELLIGENZA ARTIFICIALE PER L'INGEGNERIA BIOMEDICA (EX ROBOTICA MEDICA)

a.a. 2025/2026

Prof. Roberto Pirrone

CHILAB

Installazione e struttura dei moduli

Il framework ML di riferimento per Python

Installazione

Terminale

```
# pip
pip install scikit-learn

# conda
conda install scikit-learn

# aggiornamento
pip install --upgrade scikit-learn

# verifica
python -c "import sklearn; print(sklearn.__version__)"

# dipendenze auto-installate:
# numpy >= 1.17  scipy >= 1.3  joblib >= 1.1
```

■ Convenzione import

Importazione convenzionale:

```
import sklearn
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model  import LogisticRegression
```

Struttura dei moduli principali

- `sklearn.datasets` — dataset di esempio e generatori sintetici
- `sklearn.preprocessing` — scalatura, codifica, normalizzazione
- `sklearn.impute` — imputazione dei valori mancanti
- `sklearn.feature_selection` — selezione delle feature rilevanti
- `sklearn.decomposition` — PCA, NMF, ICA, TruncatedSVD
- `sklearn.linear_model` — regressione lineare/logistica, Ridge, Lasso
- `sklearn.tree` — alberi decisionali
- `sklearn.ensemble` — Random Forest, GradientBoosting, AdaBoost
- `sklearn.svm` — Support Vector Machine (SVC, SVR, LinearSVC)
- `sklearn.neighbors` — KNN classificatore e regressore
- `sklearn.naive_bayes` — classificatori bayesiani (GNB, MultinomialNB)
- `sklearn.cluster` — K-Means, DBSCAN, AgglomerativeClustering
- `sklearn.model_selection` — CV, GridSearchCV, train_test_split
- `sklearn.metrics` — metriche di valutazione e matrici
- `sklearn.pipeline` — Pipeline e ColumnTransformer

API unificata: fit, transform, predict

Tutti gli estimatori seguono la stessa interfaccia

Metodi fondamentali

Metodo	Chi	Descrizione
`fit(X, y)`	tutti	apprende parametri dai dati
`predict(X)`	clf / regressori	predice classe o valore
`predict_proba(X)`	classificatori	probabilità per ogni classe
`transform(X)`	trasformatori	trasforma i dati
`fit_transform(X,y)`	trasformatori	fit + transform in un passo
`fit_predict(X)`	clustering	fit + assegna i cluster
`score(X, y)`	tutti	metrica predefinita
`get_params()`	tutti	restituisce iperparametri
`set_params(**p)`	tutti	imposta iperparametri

■ Convenzioni

X: array-like (n_samples, n_features)
y: array-like (n_samples,) — omesso per apprendimento non supervisionato
Tutti i metodi restituiscono self per consentire il metodo chaining.

Flusso tipico fit → transform → predict

Python

```python
from sklearn.preprocessing   import StandardScaler
from sklearn.linear_model    import LogisticRegression
from sklearn.datasets        import load_iris
from sklearn.model_selection import train_test_split

X, y = load_iris(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Trasformatore ■ fit SOLO su training set
scaler = StandardScaler()
X_tr_s = scaler.fit_transform(X_tr)  # fit + transform
X_te_s = scaler.transform(X_te)      # solo transform!

# Classificatore
clf = LogisticRegression(max_iter=200)
clf.fit(X_tr_s, y_tr)

y_pred  = clf.predict(X_te_s)         # classi predette
y_prob  = clf.predict_proba(X_te_s)   # prob. per classe
acc     = clf.score(X_te_s, y_te)
print(f'Accuracy: {acc:.3f}')         # 0.967
```

■ Data leakage

fit_transform() su test set causa data leakage: il modello vede statistiche del test set. Usare sempre fit() su train, transform() su test.

Attributi del modello dopo fit()

Parametri appresi — convenzione: nome con underscore finale (attr_)

Parametri appresi: esempi pratici

Python

```python
# ■■ Regressione lineare ■■■■■■■■■■■■■■■■■■■■■■■■
from sklearn.linear_model import LinearRegression
import numpy as np
X = np.array([[1],[2],[3],[4],[5]])
y = np.array([2.0, 4.1, 5.0, 4.2, 5.1])
reg = LinearRegression().fit(X, y)
reg.coef_           # [0.74]   — pesi
reg.intercept_      # 1.90     — bias
reg.n_features_in_  # 1

# ■■ Classificatore ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
from sklearn.linear_model import LogisticRegression
clf = LogisticRegression().fit(X_tr_s, y_tr)
clf.classes_        # [0, 1, 2]
clf.coef_           # shape (3, 4) — un vettore per classe
clf.intercept_      # shape (3,)

# ■■ Clustering ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
from sklearn.cluster import KMeans
km = KMeans(n_clusters=3, random_state=0).fit(X)
km.cluster_centers_ # shape (3, n_features)
km.labels_          # cluster per ogni campione
km.inertia_         # somma distanze^2 dai centroidi
km.n_iter_          # iterazioni eseguite
```

Tabella degli attributi comuni

Attributo (attr_)	Disponibile in	Significato
coef_	lineari, SVM, LR	coefficienti / pesi
intercept_	modelli lineari	bias / termine noto
feature_importances_	ensemble, tree	importanza feature
classes_	classificatori	etichette di classe viste
n_features_in_	tutti (>=0.24)	# feature in input
n_iter_	iterativi	# iterazioni convergenza
cluster_centers_	KMeans	coordinate centroidi
labels_	clustering	cluster di ogni campione
inertia_	KMeans	somma dist. quadratiche
components_	PCA, NMF, ICA	componenti / basi
explained_variance_ratio_	PCA	varianza per componente
support_vectors_	SVM	vettori di supporto
estimators_	ensemble	stimatori base addestrati
mean_	SimpleImputer	media per imputazione
statistics_	SimpleImputer	statistica usata per fill

■ Before fit()

Accedere a coef_ prima di fit() genera NotFittedError.
check_is_fitted(estimator) per verifica programmatica.

Feature selection

sklearn.feature_selection — ridurre la dimensionalità mantenendo l'informazione

Classi principali

Classe	Strategia	Supervisiona
`VarianceThreshold`	rimuove feature a bassa varianza	No
`SelectKBest`	top-k feature per punteggio statistico	Si
`SelectPercentile`	top-% feature per punteggio	Si
`SelectFromModel`	feature con importanza >= soglia	Si
`RFE`	Recursive Feature Elimination	Si
`RFECV`	RFE con cross-validation automatica	Si
`SequentialFeatureSelector`	forward/backward greedy search	Si

Funzioni di scoring per SelectKBest

Funzione	Tipo	Uso
`chi2`	classif.	feature categoriche, y categorica
`f_classif`	classif.	ANOVA F-test (feature continue)
`mutual_info_classif`	classif.	informazione mutua (non-lineare)
`f_regression`	regress.	correlazione lineare con y
`mutual_info_regression`	regress.	informazione mutua con y continua

Esempi di utilizzo

Python

```python
from sklearn.feature_selection import (
    VarianceThreshold, SelectKBest, f_classif,
    SelectFromModel, RFE)
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm      import SVR

# 1. Rimuovi feature a varianza quasi nulla
sel = VarianceThreshold(threshold=0.01)
X_red = sel.fit_transform(X_train)
sel.get_support()        # maschera booleana

# 2. Top-5 feature per ANOVA F-test
skb = SelectKBest(f_classif, k=5)
X_k5 = skb.fit_transform(X_train, y_train)
skb.scores_              # punteggi per feature

# 3. Selezione tramite importanza Random Forest
sfm = SelectFromModel(RandomForestClassifier(n_estimators=100))
sfm.fit(X_train, y_train)
X_sfm = sfm.transform(X_train)

# 4. RFE con SVM: mantieni le top 10 feature
rfe = RFE(estimator=SVR(kernel='linear'), n_features_to_select=10)
rfe.fit(X_train, y_train)
rfe.ranking_             # 1 = selezionata
rfe.support_             # maschera booleana
```

Preprocessing e imputazione

sklearn.preprocessing · sklearn.impute

Principali classi di preprocessing

Classe	Trasformazione
`StandardScaler`	z-score: (x-mean)/std — media 0, dev.std. 1
`MinMaxScaler`	scala in [feature_range], default [0, 1]
`RobustScaler`	usa mediana e IQR — robusto agli outlier
`Normalizer`	normalizza ogni campione a norma unitaria
`MaxAbsScaler`	scala per il valore assoluto massimo
`PowerTransformer`	Box-Cox / Yeo-Johnson per gaussianizzare
`QuantileTransformer`	mappa su distribuzione uniforme o normale
`LabelEncoder`	encode etichette intere 0..n_classes-1
`OrdinalEncoder`	encode feature categoriche ordinali
`OneHotEncoder`	encode categoriche → vettore binario sparso
`Binarizer`	soglia: feature >= threshold → 1, else 0
`PolynomialFeatures`	genera interazioni e potenze
`FunctionTransformer`	applica una funzione arbitraria numpy

Principali classi di imputazione

Classe	Strategia
`SimpleImputer`	mean / median / most_frequent / constant
`KNNImputer`	valore medio dei k vicini più simili
`IterativeImputer`	modello regressivo iterativo (MICE)
`MissingIndicator`	aggiunge feature binaria per NaN

Esempi di codice

Python

```python
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute         import SimpleImputer, KNNImputer
import numpy as np

# ■■ Scaling ■■■
```

Esempio sinottico: Clustering

sklearn.cluster — apprendimento non supervisionato

Principali algoritmi disponibili

Classe	Tipo	Parametri chiave
`KMeans`	centroide	n_clusters, init, n_init
`MiniBatchKMeans`	centroide	n_clusters, batch_size
`DBSCAN`	densità	eps, min_samples
`HDBSCAN`	densità hier.	min_cluster_size
`AgglomerativeClustering`	gerarchico	n_clusters, linkage
`SpectralClustering`	grafo	n_clusters, affinity
`GaussianMixture`	prob. (EM)	n_components, covariance_type
`MeanShift`	densità	bandwidth
`Birch`	incrementale	threshold, n_clusters

■ Note

KMeans usa fit_predict(X) oppure fit(X) + labels_.
DBSCAN e HDBSCAN non richiedono n_clusters: -1 = rumore.
GaussianMixture usa predict() e predict_proba() come i classificatori.

Esempio completo con KMeans

Python

```python
from sklearn.cluster    import KMeans, DBSCAN
from sklearn.datasets   import make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn.metrics    import silhouette_score
import matplotlib.pyplot as plt

# Dati sintetici: 300 campioni, 4 cluster
X, y_true = make_blobs(n_samples=300, centers=4,
                       cluster_std=0.6, random_state=0)

# Preprocessing
X_s = StandardScaler().fit_transform(X)

# K-Means
km = KMeans(n_clusters=4, n_init=10, random_state=42)
labels = km.fit_predict(X_s)       # o km.fit(X_s).labels_

print(km.inertia_)                 # inerzia
print(silhouette_score(X_s, labels))  # ~0.72

# DBSCAN (no n_clusters)
db = DBSCAN(eps=0.3, min_samples=5).fit(X_s)
n_cluster = len(set(db.labels_)) - (1 if -1 in db.labels_ else 0)
n_noise   = list(db.labels_).count(-1)

# Elbow method per scegliere k
inertias = [KMeans(n_clusters=k, n_init=10).fit(X_s).inertia_
            for k in range(2, 10)]
plt.plot(range(2,10), inertias, 'bo-'); plt.show()
```

Esempio sinottico: Classificatore

sklearn — apprendimento supervisionato (output discreto)

Principali classificatori disponibili

Classe	Modulo
`LogisticRegression`	linear_model
`LinearSVC / SVC`	svm
`KNeighborsClassifier`	neighbors
`DecisionTreeClassifier`	tree
`RandomForestClassifier`	ensemble
`GradientBoostingClassifier`	ensemble
`HistGradientBoostingClassifier`	ensemble (veloce, NaN-ok)
`AdaBoostClassifier`	ensemble
`GaussianNB`	naive_bayes
`MLPClassifier`	neural_network

Pipeline con preprocessore

Python – Pipeline

```python
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier

pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('clf',    RandomForestClassifier(n_estimators=100))
])
pipe.fit(X_train, y_train)
pipe.score(X_test, y_test)  # applica pipeline intera
```

Esempio completo con Random Forest

Python

```python
from sklearn.datasets        import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing   import StandardScaler
from sklearn.ensemble        import RandomForestClassifier
from sklearn.metrics         import classification_report
from sklearn.metrics         import ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# Carica dataset
X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

# Scaling + modello
sc  = StandardScaler()
X_tr_s = sc.fit_transform(X_tr)
X_te_s = sc.transform(X_te)

clf = RandomForestClassifier(n_estimators=200, random_state=42)
clf.fit(X_tr_s, y_tr)

# Valutazione
y_pred = clf.predict(X_te_s)
print(classification_report(y_te, y_pred))

# Visualizzazione matrice di confusione
ConfusionMatrixDisplay.from_estimator(
    clf, X_te_s, y_te, cmap='Blues')
plt.show()

# Importanza feature
importances = clf.feature_importances_
```

Esempio sinottico: Regressore

sklearn — apprendimento supervisionato (output continuo)

Principali regressori disponibili

Classe	Modulo / Note
`LinearRegression`	linear_model — OLS classico
`Ridge`	linear_model — L2 regularization
`Lasso`	linear_model — L1 (sparsità)
`ElasticNet`	linear_model — L1 + L2
`BayesianRidge`	linear_model — prior bayesiano
`SVR`	svm — kernel rbf/linear/poly
`KNeighborsRegressor`	neighbors
`DecisionTreeRegressor`	tree
`RandomForestRegressor`	ensemble
`GradientBoostingRegressor`	ensemble
`HistGradientBoostingRegressor`	ensemble — NaN-ok, veloce
`MLPRegressor`	neural_network

Esempio completo con Ridge regression

Python

```python
from sklearn.datasets        import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.preprocessing   import StandardScaler
from sklearn.linear_model    import Ridge, Lasso
from sklearn.metrics         import mean_squared_error, r2_score
import numpy as np

X, y = load_diabetes(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42)

sc = StandardScaler()
X_tr_s = sc.fit_transform(X_tr)
X_te_s = sc.transform(X_te)

reg = Ridge(alpha=1.0)
reg.fit(X_tr_s, y_tr)
y_pred = reg.predict(X_te_s)

rmse = np.sqrt(mean_squared_error(y_te, y_pred))
r2   = r2_score(y_te, y_pred)
print(f'RMSE: {rmse:.2f}  R2: {r2:.3f}')

# Confronto alpha con più modelli
for alpha in [0.01, 0.1, 1.0, 10.0, 100.0]:
    r = Ridge(alpha=alpha).fit(X_tr_s, y_tr)
    sc_r2 = r2_score(y_te, r.predict(X_te_s))
    print(f'alpha={alpha:6.2f}  R2={sc_r2:.3f}')

# Lasso: mostra coef. azzerati (feature selection)
las = Lasso(alpha=0.1).fit(X_tr_s, y_tr)
print('Coeff. non-zero:', np.sum(las.coef_ != 0))
```

Cross-validation e selezione degli iperparametri

sklearn.model_selection

Strategie di splitting e CV

Classe / Funzione	Descrizione
`train_test_split()`	split singolo train/test
`KFold(n_splits=k)`	k-fold standard
`StratifiedKFold`	k-fold con proporzioni classi invariate
`RepeatedKFold`	k-fold ripetuto n volte
`LeaveOneOut (LOO)`	k=n campioni
`ShuffleSplit`	split casuali ripetuti
`GroupKFold`	evita leakage tra gruppi
`TimeSeriesSplit`	fold temporalmente ordinati
`cross_val_score()`	CV semplice, restituisce array score
`cross_validate()`	CV con più metriche + fit_time
`cross_val_predict()`	predizioni out-of-fold
`GridSearchCV`	griglia esaustiva di iperparametri
`RandomizedSearchCV`	campionamento casuale degli iperparametri
`HalvingGridSearchCV`	griglia con successive halving

Esempio completo cross-val + GridSearch

Python

```python
from sklearn.model_selection import (
    cross_val_score, cross_validate,
    GridSearchCV, StratifiedKFold)
from sklearn.ensemble  import RandomForestClassifier
from sklearn.datasets  import load_iris
import numpy as np

X, y = load_iris(return_X_y=True)

# ■■ CV semplice ■■■
```

Metriche per la classificazione

sklearn.metrics — valutazione dei modelli supervisioned (output discreto)

Metriche scalari

Funzione	Descrizione
`accuracy_score`	proporzione di predizioni corrette
`balanced_accuracy_score`	accuracy bilanciata per classi sbilanciate
`precision_score`	TP / (TP + FP)
`recall_score`	TP / (TP + FN) — sensibilità
`f1_score`	media armonica di precision e recall
`fbeta_score`	f-score con peso beta su recall
`roc_auc_score`	area sotto la curva ROC
`average_precision_score`	area sotto curva Precision-Recall
`log_loss`	cross-entropy (usa probabilità)
`matthews_corrcoef`	MCC — robusto a classi sbilanciate
`cohen_kappa_score`	kappa di Cohen

■ average

Parametro average per metriche multiclasse:
'binary' | 'micro' | 'macro' | 'weighted' | 'samples'

Classi per la visualizzazione

Classe display	Cosa mostra
`ConfusionMatrixDisplay`	matrice di confusione
`RocCurveDisplay`	curva ROC + AUC
`PrecisionRecallDisplay`	curva Precision-Recall
`DetCurveDisplay`	curva DET (FPR vs FNR)

Python

```python
from sklearn.metrics import (
    classification_report, confusion_matrix,
    roc_auc_score, RocCurveDisplay,
    ConfusionMatrixDisplay, PrecisionRecallDisplay)
import matplotlib.pyplot as plt

# Report testuale (precision/recall/f1 per classe)
print(classification_report(y_te, y_pred,
      target_names=['maligno','benigno']))

# Matrice di confusione
ConfusionMatrixDisplay.from_predictions(
    y_te, y_pred, cmap='Blues')
plt.show()

# Curva ROC (binaria)
RocCurveDisplay.from_estimator(clf, X_te_s, y_te)
plt.show()

# ROC multiclasse (OvR)
y_score = clf.predict_proba(X_te_s)
auc = roc_auc_score(y_te, y_score,
        multi_class='ovr', average='macro')
print(f'ROC-AUC (macro): {auc:.3f}')
```

Metriche per regressione e clustering

sklearn.metrics — valutazione quantitativa e non supervisionata

Metriche per la regressione

Funzione	Formula / Descrizione
`mean_squared_error`	MSE = mean((y-y_hat)^2)
`root_mean_squared_error`	RMSE = sqrt(MSE) (sklearn >=1.4)
`mean_absolute_error`	MAE = mean(|y-y_hat|)
`mean_absolute_percentage_error`	MAPE = mean(|y-y_hat|/|y|)*100
`r2_score`	R^2 = 1 - SS_res/SS_tot in [-inf,1]
`explained_variance_score`	simile R^2 ma non penalizza bias
`mean_squared_log_error`	per target con distribuzione skewed
`median_absolute_error`	robusto agli outlier
`max_error`	massimo errore assoluto
`d2_absolute_error_score`	D^2 analogo a R^2 per MAE

Python – Regressione

```
from sklearn.metrics import (
    mean_squared_error, mean_absolute_error,
    r2_score, PredictionErrorDisplay)
import numpy as np, matplotlib.pyplot as plt

rmse = np.sqrt(mean_squared_error(y_te, y_pred))
mae  = mean_absolute_error(y_te, y_pred)
r2   = r2_score(y_te, y_pred)
print(f'RMSE={rmse:.2f}  MAE={mae:.2f}  R2={r2:.3f}')

# Grafico residui
PredictionErrorDisplay.from_estimator(
    reg, X_te_s, y_te, kind='residual_vs_predicted')
plt.show()
```

Metriche per il clustering

Funzione	Note
`silhouette_score`	[-1, 1] — non serve ground truth
`silhouette_samples`	silhouette per ogni campione
`calinski_harabasz_score`	rapporto varianza inter/intra
`davies_bouldin_score`	minimizzare — 0 = ottimo
`adjusted_rand_score`	serve ground truth — corretto per caso
`rand_score`	serve ground truth
`adjusted_mutual_info_score`	serve ground truth
`normalized_mutual_info_score`	serve ground truth
`fowlkes_mallows_score`	serve ground truth
`homogeneity_score`	ogni cluster = 1 sola classe
`completeness_score`	ogni classe = 1 solo cluster
`v_measure_score`	media arm. homogeneity + completeness

Python – Clustering

```
from sklearn.metrics import (
    silhouette_score, calinski_harabasz_score,
    davies_bouldin_score, adjusted_rand_score)

# Senza ground truth
sil = silhouette_score(X_s, labels)       # ~0.72
ch  = calinski_harabasz_score(X_s, labels)
db  = davies_bouldin_score(X_s, labels)   # minimizzare

# Con ground truth
ari = adjusted_rand_score(y_true, labels) # 1=perfetto
nmi = normalized_mutual_info_score(y_true, labels)
```

Riepilogo e Risorse

scikit-learn in una pagina

Argomenti trattati

- **Installazione:** pip / conda install scikit-learn
- **Moduli:** preprocessing, impute, feature_selection, cluster, metrics…
- **API fit/predict:** fit · transform · predict · predict_proba · fit_transform · fit_predict
- **Attributi post-fit:** coef_ intercept_ labels_ feature_importances_ components_…
- **Feature selection:** VarianceThreshold · SelectKBest · SelectFromModel · RFE · RFECV
- **Preprocessing:** StandardScaler · MinMaxScaler · OneHotEncoder · PolynomialFeatures
- **Imputazione:** SimpleImputer · KNNImputer · IterativeImputer · MissingIndicator
- **Clustering:** KMeans · DBSCAN · AgglomerativeClustering · GaussianMixture
- **Classificazione:** LogisticRegression · RandomForest · SVC · GradientBoosting · MLP
- **Regressione:** LinearRegression · Ridge · Lasso · SVR · RandomForestRegressor
- **Cross-validation:** cross_val_score · GridSearchCV · StratifiedKFold · RandomizedSearchCV
- **Metriche clf:** accuracy · precision · recall · f1 · ROC-AUC · ConfusionMatrixDisplay
- **Metriche regr.:** MSE · RMSE · MAE · R^2 · PredictionErrorDisplay
- **Metriche cluster:** silhouette · calinski_harabasz · davies_bouldin · ARI · NMI

Risorse ufficiali

- **Documentazione sklearn:** https://scikit-learn.org/stable/
- **User guide completa:** https://scikit-learn.org/stable/user_guide.html
- **API reference:** https://scikit-learn.org/stable/api/index.html
- **Tutorial ufficiale:** https://scikit-learn.org/stable/tutorial/index.html
- **Esempi e gallery:** https://scikit-learn.org/stable/auto_examples/
- **Choosing the right estimator:** https://scikit-learn.org/stable/tutorial/machine_learning_map/
- **Metriche e scoring:** https://scikit-learn.org/stable/modules/model_evaluation.html
- **sklearn-compatible ecosystem:** https://scikit-learn.org/stable/related_projects.html

Prossimi passi

→ Pipeline + ColumnTransformer per pre-processing eterogeneo

→ imbalanced-learn: SMOTE e tecniche per classi sbilanciate

→ Optuna / scikit-optimize per ottimizzazione bayesiana degli iper.

→ SHAP + sklearn: spiegabilità dei modelli

→ XGBoost / LightGBM / CatBoost: boost avanzati sklearn-compatibili